

Aforo Loadgen

User Guide for the Platform Team

aforo-loadgen is a Go CLI for load-testing the Aforo NextGen platform's ingestion pipeline. This guide walks any developer with a fresh laptop through installing the tool, connecting it to Aforo, and running their first end-to-end test — including reading the report, reproducing failures, and adding custom scenarios.

Version	v0.1.0
Released	2026-04-30
Tool repo	github.com/aforoi/aforo-nextgen-loadgen (public)
Distribution	Homebrew tap: aforoi/tap (public)
Audience	Aforo platform engineers (Go/Java/Spring background)
Supported OS	macOS (Intel + Apple Silicon), Linux (amd64 + arm64)
Source of truth	This guide + repo /docs

1. What This Tool Is (and Isn't)

Aforo Loadgen is the platform team's standard tool for verifying the ingestion pipeline before code reaches production. It seeds realistic test tenants via real REST APIs, generates events for all four product types across all 16 ingestion paths, then validates billing math end-to-end.

It is for

- Verifying your changes to **usage-ingestor**, **analytics-service**, **billing-platform**, or **billing-service** before merge.
- Catching cross-tenant leakage and tenant-isolation regressions automatically.
- Validating that all 6 pricing models (PER_UNIT, FLAT_RATE, PERCENTAGE, INCLUDED_QUOTA, GRADUATED, VOLUME_TIERED) compute the right invoice amount under load.
- Catching dunning, lifecycle, payment, tax, ERP, and credit-note regressions.
- Reproducing flaky bugs deterministically via seeded RNG.
- Integrating into CI as a gate on every PR (already wired in 4 service repos).

It is not

- A unit-test replacement. Unit tests still gate compilation; loadgen gates behavior.
- A profiling tool. Use pprof for CPU/heap analysis (loadgen exposes /debug/pprof when --pprof-port is set, but interpretation is on you).
- A chaos engineering platform. The chaos package supports targeted fault injection during a run, but it is not Litmus or Chaos Mesh.
- Customer-facing. It runs against test tenants in your dev or staging environment, never production.

2. Prerequisites — Per Laptop

Set these up once on each developer machine. First-time total: ~30 minutes if you don't already have Docker. ~5 minutes if you do.

2.1 Operating system

macOS (Intel or Apple Silicon) is the supported primary. Linux works via Linuxbrew or direct binary download. Windows users should use WSL2 + Linuxbrew, or Go install path (Section 3.2).

2.2 Required software

Tool	Why	Install command
Homebrew	Tool installation	<code>/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"</code>
Docker Desktop	Local Aforo backend stack	<code>brew install --cask docker</code> OR download from docker.com
git	Cloning Aforo repos	<code>brew install git</code>
gh (GitHub CLI)	Optional, but useful for CI checks	<code>brew install gh && gh auth login</code>
Java 17+	Aforo backend services	<code>brew install openjdk@17</code>
Maven 3.9+	Building Aforo backend	<code>brew install maven</code>

2.3 GitHub access

Each developer needs a GitHub account that is a member of the **aforoai** organization. Repository visibility:

Repository	Visibility	Access needed
aforoai/aforo-nextgen-loadgen	Public	None — anyone can read, fork, file issues
aforoai/homebrew-tap	Public	None — brew install works without auth

Repository	Visibility	Access needed
aforoai/aforo-nextgen-docker	Private	Read access required (your org admin grants)
aforoai/aforo-nextgen-* (services)	Private	Only if you'll run services from source

Action for new dev: ask in #platform Slack to be added to the aforoai org with at least Read access on aforo-nextgen-docker.

3. Installation

3.1 Recommended: Homebrew

On macOS or Linux with Linuxbrew:

```
brew tap aforoai/tap
brew install aforoai/tap/loadgen
aforo-loadgen version
```

You should see: aforo-loadgen v0.1.0 (commit 7b4893c, built 2026-04-30T...)

Updating later: brew update && brew upgrade aforoai/tap/loadgen

3.2 Alternative: Go install

If you have Go 1.22+ already and prefer pinning to a specific version:

```
go install github.com/aforoai/aforo-nextgen-loadgen/cmd/aforo-loadgen@v0.1.0
aforo-loadgen version
```

3.3 Alternative: Direct binary download

For environments where Homebrew or Go aren't available, download the pre-built binary for your platform from the GitHub Releases page:

```
# https://github.com/aforoai/aforo-nextgen-loadgen/releases/tag/v0.1.0
# Pick the right asset:
# aforo-loadgen_Darwin_arm64.tar.gz (Apple Silicon Macs)
# aforo-loadgen_Darwin_x86_64.tar.gz (Intel Macs)
# aforo-loadgen_Linux_arm64.tar.gz (Linux ARM)
# aforo-loadgen_Linux_x86_64.tar.gz (Linux Intel)

tar xzf aforo-loadgen_Darwin_arm64.tar.gz
sudo mv aforo-loadgen /usr/local/bin/
aforo-loadgen version
```

Verify checksums.txt from the same release page if you care about supply-chain integrity.

4. First-Time Setup — Get to a Green Doctor

`aforo-loadgen doctor` is the pre-flight check. Until it exits 0, no scenario will run successfully. This section gets you there.

4.1 Start the local Aforo backend

Loadgen needs a target Aforo stack to test against. The simplest is the local Docker stack:

```
# Clone the docker repo (one-time)
cd ~/dev # or wherever you keep your repos
git clone git@github.com:aforoai/aforo-nextgen-docker.git
cd aforo-nextgen-docker

# Read SETUP.md for full prerequisites (Java/Maven/etc.)
open SETUP.md

# Start everything (Docker + 10 backend services + frontends)
./start-platform.sh
```

The stack takes 30-90 seconds on a warm cache, longer on first run (Keycloak alone takes ~60s).

Don't have access to `aforo-nextgen-docker`? See Section 2.3 — you need org admin to grant Read access. While you wait, you can target your team's `--target staging` environment if your team has one configured.

4.2 Set your admin token

Loadgen authenticates to Aforo using an admin JWT. The token lives in the `AFORO_ADMIN_TOKEN` environment variable.

Where to get the token:

- **Local stack (`target=local`):** ask in #platform Slack for the dev admin token, or generate one via Keycloak at `http://localhost:8180 (admin/admin)` after the stack is up.
- **Staging (`target=staging`):** ask your team lead. The token rotates periodically; treat it like a credential.
- **Perf cluster (`target=perf-aws`):** only platform leads have this; not for individual dev use.

```
# Add this to your ~/.zshrc or ~/.bashrc so it persists
export AFORO_ADMIN_TOKEN='eyJhbGciOiJSUzI1NiIs...'
source ~/.zshrc
```

4.3 Run doctor

```
aforo-loadgen doctor --target local
```

A green doctor reports all 9 services reachable, auth valid, and infrastructure (PostgreSQL, Redis, Kafka, ClickHouse) healthy. If anything is red, fix it before moving on — every failure has an actionable remedy printed inline.

4.4 Run your first scenario

```
aforo-loadgen e2e \  
--scenario scenarios/crawl-e2e.yaml \  
--target local \  
--include-billing
```

This chains seed → run → validate → report → cleanup. Total wall time on a healthy stack: ~8 minutes. The output directory is printed at the end (something like `out/run-2026-...`); open `report.html` in that directory to see results.

If this works, you're done with setup. The rest of this guide is reference.

5. Common Workflows

Each subsection answers a specific question with a copy-pasteable command. Pick the workflow that matches what you're doing.

5.1 "I changed usage-ingestor — does it still work?"

Local Docker stack must be running. Catches event count mismatches, billing math drift, stale-key cache invalidation, and lifecycle regressions in ~8 minutes.

```
aforo-loadgen e2e \  
--scenario scenarios/crawl-e2e.yaml \  
--target local \  
--include-billing
```

5.2 "I want to load-test against staging before promoting to prod."

Walk-phase scenario: 50 tenants across 8 archetypes, 2K TPS for 30 min. Validates per-tenant fairness (no noisy neighbor), tenant isolation, and cache hit ratio.

```
export AFORO_STAGING_TOKEN='...'  
aforo-loadgen e2e \  
--scenario scenarios/walk-realistic-50t.yaml \  
--target staging \  
--duration 30m \  
--include-billing \  
--include-lifecycle
```

5.3 "I added or modified a pricing model."

Matrix-billing scenario exercises all 6 pricing models × 3 billing modes (30 archetypes). Validates exact invoice \$ to within 0.001%.

```
aforo-loadgen e2e \  
--scenario scenarios/matrix-billing.yaml \  
--target local \  
--include-billing
```

5.4 "I touched API key revocation or BillingHierarchyEnricher."

Stale-keys scenario: tests that cancelled-subscription keys reject within 60s, explicit-revoke keys reject immediately, with zero false positives.

```
aforo-loadgen e2e \  
--scenario scenarios/ci-stale-keys.yaml \  
--target local
```


5.5 "I want to test a specific gateway adapter (Kong, AWS, Apigee, etc.)."

Walk scenario hits all 16 ingestion paths weighted. To focus on one, copy walk-realistic and set the path's weight to 1.0 (others 0). Validate the YAML before running.

```
cp scenarios/walk-realistic-50t.yaml scenarios/my-kong-only.yaml
# edit ingestion_paths so kong=1.0, others=0
aforo-loadgen scenarios validate scenarios/my-kong-only.yaml
aforo-loadgen e2e --scenario scenarios/my-kong-only.yaml --target local
```

5.6 "I want to reproduce a flaky test failure."

Every run with the same scenario + manifest + seed produces identical events. Replay re-sends an exact past event sequence.

```
aforo-loadgen replay \
--run-output out/<run-id-from-failed-run> \
--target local
```

5.7 "I want to test payment processing / dunning / Stripe integration."

Requires STRIPE_TEST_SECRET_KEY env var. Tests payment success, decline, insufficient funds, and full dunning escalation to SUSPEND/CANCEL.

```
export STRIPE_TEST_SECRET_KEY='sk_test_...'
aforo-loadgen payments \
--scenario scenarios/payments-stripe-test.yaml \
--target local \
--stripe-mode test
```

6. Reading the Results

After any run, check out `/<run-id>/`. The most useful files:

File	What it tells you
report.html	Open this first. Self-contained HTML (no internet needed). Latency histograms, per-tenant breakdown, per-archetype billing accuracy, validation pass/fail summary, per-negative-path table.
validation.json	Per-check pass/fail with exact details. Useful for grepping in CI scripts. Each failed check includes the file:line of the assertion that failed and the actual vs expected values.
run.json	Raw run summary: events sent/succeeded/failed, latency percentiles, per-tenant counts, archetype distribution, total wall time.
events.jsonl	First 1000 generated events with their negative-path tags. Useful for "why was this event shaped this way?" debugging.
transitions.jsonl	Lifecycle transitions log (only if <code>--include-lifecycle</code> was used). Records every upgrade/downgrade/migrate/etc. with expected pro-ration credit.
manifest.json	The seed data the run used: tenant IDs, customer IDs, API keys, archetype mapping. Required input to validate and replay subcommands.
latencies.hdr	HDR histogram binary. Use a HDR histogram viewer to compute exact percentiles beyond p99.

Highest-signal panels in report.html

- **Per-archetype billing accuracy table.** If any row shows non-zero drift, your billing math has a bug for that pricing × billing combination.
- **Latency histogram (p50 / p95 / p99).** Sustained p99 above 500ms during steady-state load is a regression signal.
- **Per-tenant fairness check.** Stddev of p99 across tenants > 30% of mean = noisy-neighbor bug. One tenant is impacting others.
- **Negative-path table.** All 6 categories (late, future, malformed, wrong_auth, stale_key, oversize) should show 100% rejection at the expected HTTP code.
- **Cross-tenant leakage check.** Must always be 0. Any non-zero is a critical security bug.

7. Scenario Reference

All built-in scenarios live in `scenarios/*.yaml` in the loadgen repo. List them with `aforo - loadgen scenarios list`.

Scenario	Use case	TPS × duration	Target
ci-smoke	Generic 1-tenant 4-product smoke. Used by CI on every PR.	50 × 60s	local / ci
ci-mcp-only	MCP_SERVER product type only. Used in usage-ingestor PR CI.	50 × 60s	local / ci
ci-billing	Billing matrix subset for billing-platform PR CI.	100 × 60s	local / ci
ci-payments-mock	Payments + tax with mock engines. Billing-service PR CI.	100 × 60s	local / ci
ci-stale-keys	Focused stale-key revocation cascade test.	50 × 60s	local / ci
crawl-e2e	4 archetypes × 4 product types. The standard local smoke.	50 × 5m	local
crawl-mcp-server	MCP-only crawl with deeper coverage.	50 × 5m	local
walk-realistic-50t	50 tenants Pareto, 8 archetypes, all 16 ingestion paths. Time-of-day sine.	2000 × 24h	staging
matrix-billing	Exhaustive 6 pricing × 3 billing combinations (30 archetypes).	500 × 30m	local / staging
lifecycle-stress	Lifecycle transitions at full rate (upgrade/downgrade/migrate/dunning).	1000 × 4h	staging
payments-stripe-test	Full payment flow with Stripe test mode.	100 × 30m	local / staging
erp-sync-validation	ERP sync coverage across QuickBooks / Xero / NetSuite / Custom.	200 × 30m	staging
multi-currency	USD / EUR / GBP customer mix with FX strategy validation.	200 × 30m	staging

Scenario	Use case	TPS × duration	Target
wallet-lifecycle	PREPAID / HYBRID heavy. Validates hold expiry scheduler.	500 × 1h	staging
run-15k-7day	Production-scale soak. Perf cluster only.	15000 × 7d	perf-aws

8. Troubleshooting

If you hit one of these errors, here's the fix:

Error	Fix
doctor: usage-ingestor not reachable at localhost:8084	The Aforo backend isn't running. <code>cd aforo-nextgen-docker && ./start-platform.sh</code> . Wait 60-90s for Keycloak.
doctor: auth invalid	AFORO_ADMIN_TOKEN is unset, expired, or wrong for this target. See Section 4.2 for token sourcing.
seed: tenant intersection produced empty manifest	Manifest is from a different scenario. Re-run seed first, or use the e2e command which seeds automatically.
validate: stale_key zero-false-positives FAILED	Real bug. The BillingHierarchyEnricher Redis cache isn't invalidating on key revocation. File a bug.
validate: per_archetype_billing_match FAILED	Real bug in pricing or rate plan. The per-archetype drift report in validation.json shows which combination is wrong.
run: backpressure activated for 30s+	Downstream is overloaded. Lower <code>scenario.target_tps</code> , or look at <code>/metrics</code> endpoint to find the bottleneck.
scenarios validate: weights do not sum to 1.0	Your custom scenario has <code>archetype.weight</code> or <code>product_mix</code> weights summing outside [0.999, 1.001]. The validator points to the exact field.
replay: events.jsonl not found	Replay needs an <code>out/<run-id></code> directory from a prior run, not a scenario file. Pass <code>--run-output</code> , not <code>--scenario</code> .
brew install fails with 404	Tap or release repo isn't accessible. Check <code>brew tap aforoi/tap</code> printed success. Run <code>brew untap aforoi/tap && brew tap aforoi/tap</code> if stale.

Error	Fix
all scenarios fail with 503 from kafka	Local Kafka container is unhealthy. docker compose restart kafka in the aforo-nextgen-docker repo, wait 30s.

9. Adding a Custom Scenario

When the built-in scenarios don't cover what you need, copy one and modify. The schema is documented exhaustively in `docs/scenario-schema.md` in the loadgen repo.

9.1 Workflow

```
# 1. Copy a similar scenario
cp scenarios/crawl-e2e.yaml scenarios/my-test.yaml

# 2. Edit it (vim/cursor/your-editor)
# Common knobs: tenants.count, target_tps, duration,
# product_mix, ingestion_paths, lifecycle.enabled,
# payments.enabled, assertions.*

# 3. Validate strictly before running
aforo-loadgen scenarios validate scenarios/my-test.yaml

# 4. Show what was parsed
aforo-loadgen scenarios show my-test

# 5. Run it
aforo-loadgen e2e \
--scenario scenarios/my-test.yaml \
--target local \
--include-billing
```

9.2 Common patterns

- **Test one ingestion path:** set `ingestion_paths` to weight 1.0 on the target path, 0 on others.
- **Test one product type:** set `product_mix` to 1.0 on the target type, 0 on others.
- **Test one pricing model:** in `tenants.archetypes`, set weight 1.0 on an archetype with the target `pricing_model`.
- **Tighten or loosen assertions:** `assertions.invoice_revenue_drift_pct_max` and `friends`.
- **Reproducibility:** set `seed: <int>` at the top level. Same seed + same manifest = identical event sequence.
- **Custom archetypes:** define your own pricing config + state mix to test specific tenant shapes (e.g. 95% TRIALING tenants for trial-conversion stress).

9.3 Promoting your scenario into CI

If your custom scenario should run on every PR for a particular service, follow the template in `docs/ci-integration.md` in the loadgen repo. The 4 wired-in CI jobs are good examples — you'll add a YAML stanza to the relevant service's `.github/workflows/ci.yml`.

10. CI Integration (Already Wired)

PRs to these four Aforo service repos already run a 60-second loadgen smoke test as a required check. If your PR fails the loadgen - smoke check, the failure is shown in the PR's check details with a link to the run output.

Service repo	Scenario used	Why
aforo-nextgen-usage-ingestor-service	ci-mcp-only.yaml	MCP product type is the highest-throughput path; catches MCP-specific regressions.
aforo-nextgen-analytics-service	ci-smoke.yaml	Generic ingestion + ClickHouse write health.
aforo-nextgen-billing-service	ci-payments-mock.yaml	Billing pipeline + payments with mocked tax/ERP. Catches billing math drift.
aforo-billing-platform	ci-billing.yaml	Billing matrix subset across archetypes. Catches per-pricing-model regressions.

10.1 Debugging a failing CI run

If loadgen - smoke fails on your PR:

```
# 1. Check out your branch locally
git checkout my-feature-branch

# 2. Start your local stack
cd ../aforo-nextgen-docker && ./start-platform.sh && cd -

# 3. Build your service from source against your branch
# 4. Run the same scenario the CI ran
aforo-loadgen e2e \
--scenario scenarios/ci-smoke.yaml \
--target local \
--include-billing

# 5. Open the report.html and find the failing check
```


11. Things to Never Do

- Run `walk-realistic-50t.yaml` against `--target local`. A laptop cannot sustain 2K TPS for 24h. The CLI requires `--i-know-what-im-doing` if you try.
- Run `run-15k-7day.yaml` from anywhere except the dedicated AWS perf cluster. The CLI prompts for explicit confirmation; type 'no' if you weren't expecting it.
- Edit `manifest.json` by hand. It's contractual with the validator. Hand-edits cause silent drift between what the generator sent and what the validator expects.
- Skip doctor "to save time." A broken target wastes a full run before you find out.
- Run loadgen against production. Even read-only operations create test entities; never point `--target` at a prod URL.
- Bump `scenario.target_tps` beyond what your environment can handle and assume the loadgen will be the bottleneck. Use backpressure logs to confirm.
- Commit a custom scenario with a hardcoded admin token in it. Tokens come from environment variables only.
- Disable assertion checks (`assertions.events_lost_max`, etc.) to make a flaky run pass. Find the flake's root cause instead.

12. Getting Help

Resource	When to use
<code>aforo-loadgen --help</code>	List all 14 subcommands.
<code>aforo-loadgen <cmd> --help</code>	Detailed help for any subcommand including all flags + examples.
<code>aforo-loadgen doctor --target local</code>	First step in any troubleshooting session.
docs/ folder in the repo	Detailed reference for scenarios, gateway adapters, payments setup, ERP onboarding, etc.
docs/scenario-schema.md	Authoritative reference for every YAML field.
github.com/aforoai/aforo-nextgen-loadgen/issues	File bugs, ask questions, request scenarios.
#platform Slack channel	Quick questions, admin token requests, staging access.
CHANGELOG.md	What changed between releases.

Quick Reference Card

First-time setup (one-shot):

```
brew tap aforoai/tap
brew install aforoai/tap/loadgen
cd ~/dev && git clone git@github.com:aforoai/aforo-nextgen-docker.git
cd aforo-nextgen-docker && ./start-platform.sh
export AFORO_ADMIN_TOKEN='...' # from #platform Slack
aforo-loadgen doctor --target local
```

Daily smoke test before pushing:

```
aforo-loadgen e2e --scenario scenarios/crawl-e2e.yaml --target local
--include-billing
```

Reproducibility:

```
aforo-loadgen replay --run-output out/<run-id> --target local
```

Last updated: 2026-04-30. Tool version v0.1.0. Built and shipped by the Aforo platform team.